Errors & Problems Register

Every defect found in a full pass over the Criation monorepo — the web storefront, admin and mobile apps, the Express API and the six shared packages. Findings are ordered by severity and each one names the file it lives in.

8	10	11	5
CRITICAL	HIGH	MEDIUM	LOW

BUILD & TYPECHECK	LINT	TESTS & CI
Pass — 10/10 workspaces	**234 errors, 2 warnings**	**None**
PAYMENT CAPABILITY	SERVER-PERSISTED DATA	DEPENDENCY ADVISORIES
None	**Auth only**	**16 (5 high)**

SUMMARY OF THE STATE OF THE CODE

The front end is substantial and well-built: 30 routes, a coherent design language, and storefront, seller, supplier, dropship, affiliate and admin consoles. The problem is that almost none of it is backed by a server. The application makes exactly four network calls in the entire codebase — login, register, logout and file upload. Cart, orders, wallet, wishlist, catalog, coupons and the admin role all live in `localStorage`, where the user controls them.

It cannot take payment. `createOrder()` runs in the browser, stamps `paymentStatus: "paid"`, invents a transaction ID and waits 1,200 ms on a `setTimeout`. The checkout card form has a test card number hard-coded into it and sends the values nowhere.

Eight findings are rated critical because each is exploitable by an unauthenticated stranger against a deployed instance, today, with no special tooling.

Index of findings

All 34 findings, most severe first. Detail for each follows in the sections after this table.

ID	PROBLEM	LOCATION
C-1	Any user can promote themselves to administrator	api/auth/login/route.ts · api/auth/register/route.ts
C-2	A public URL creates an admin account with a known password	api/seed/route.ts
C-3	The site cannot take payment — orders are marked paid in the browser	context/StoreContext.tsx · app/checkout/page.tsx
C-4	The JWT signing secret is published in the source tree	lib/auth/jwt.ts · .env.example ×2
C-5	Admin, seller and supplier consoles have no server-side authorization	app/admin/page.tsx · no middleware.ts
C-6	Write endpoints are unauthenticated and mass-assign the request body	api/products/route.ts · api/orders/route.ts
C-7	Anonymous file upload with no type, size or quota limit	api/storage/upload/route.ts
C-8	A database outage becomes an authentication bypass	api/auth/login/route.ts · api/auth/me/route.ts
H-1	Identity is read from localStorage and never verified server-side	context/StoreContext.tsx
H-2	Every first-time visitor arrives already signed in as a named user	lib/data/mockCatalog.ts
H-3	Order history exists only in the customer's browser	context/StoreContext.tsx
H-4	The wallet is free money and coupons are validated client-side	context/StoreContext.tsx
H-5	No security response headers at all	next.config.ts
H-6	The image optimizer is an open proxy	next.config.ts
H-7	No rate limiting, lockout or bot protection anywhere	all route handlers
H-8	No password policy, no verification, no password reset	api/auth/register/route.ts
H-9	Uploads to the local filesystem will not work in production	api/storage/upload/route.ts
H-10	16 dependency advisories, 5 rated high	package-lock.json
M-1	234 lint errors block any quality gate	apps/web — 41 files
M-2	Zero tests and no CI pipeline	repository root
M-3	Two backends and two type systems that disagree on price units	backend/api · packages/types · types/store.ts
M-4	No inventory control, and order numbers collide	context/StoreContext.tsx
M-5	GST is a flat 5% and the invoice is not a compliant tax invoice	context/StoreContext.tsx · lib/invoice/generateInvoice.ts

ID	PROBLEM	LOCATION
M-6	The AI concierge and AI tools are keyword matching	`components/ai/AIChatWidget.tsx · app/ai-tools/page.tsx`
M-7	Legal pages claim compliance the platform does not have	`app/legal/[slug]/page.tsx`
M-8	No transactional email or SMS	`whole app`
M-9	Product and order pages are client-only — no SSR, no SEO surface	`app/products/[id] · app/orders/[id] · no sitemap`
M-10	Currency conversion uses hardcoded rates	`context/StoreContext.tsx`
M-11	Silent database fallbacks hide outages	`lib/db/mongodb.ts · every route catch block`
L-1	The entire storefront is uncommitted	`git status, branch CT04`
L-2	A folder of personal WhatsApp photos sits at the repository root	`pp/`
L-3	The .gitignore is duplicated line-for-line	`.gitignore`
L-4	No error boundaries, and one loading state for the whole app	`apps/web/src/app`
L-5	Accessibility has not been reviewed	`storefront components`

Critical

Each of these is exploitable by an unauthenticated stranger against a deployed instance. Fix all eight before this code is reachable from the internet.

C-1 Any user can promote themselves to administrator

`apps/web/src/app/api/auth/login/route.ts:99` · `apps/web/src/app/api/auth/register/route.ts:9`

Both routes read `role` straight from the request body. Register creates the account with whatever role was posted. Login writes it onto the user *after* the password check and saves it to the database permanently:

```
// login/route.ts
if (role && user.role !== role) {
  user.role = role as Role;
  if (role === "admin") user.isAdminVerified = true;
  await user.save();
}
```

One request with `"role":"admin"` turns any account into a verified superadmin.

> **FIX**
> Never accept `role` from a client. Force `role: "customer"` on registration and delete the role-switching branch from login. Role changes belong behind an admin-only endpoint with an audit log.

C-2 A public URL creates an admin account with a known password

`apps/web/src/app/api/seed/route.ts:86`

The seed route exports both `POST` and `GET`, has no authentication, and creates `mrdiv@criation.example` with `role: "admin"`, `isAdminVerified: true` and the password `admin123`. Anyone who opens `/api/seed` on the deployed site can then sign in as an administrator. It can also be called in a loop to hammer the database.

> **FIX**
> Delete the route. Seeding belongs in a CLI script an operator runs against a database, never in an HTTP handler. If it must stay, gate it on `NODE_ENV !== "production"` plus a secret header, and remove the `GET` export.

C-3 The site cannot take payment — orders are marked paid in the browser

`apps/web/src/context/StoreContext.tsx:516` · `apps/web/src/app/checkout/page.tsx:479`

`createOrder()` runs entirely client-side: it invents an order number, sets `paymentStatus: "paid"` and a fake transaction ID, then the checkout page waits on a timer and navigates to the confirmation screen.

```
// StoreContext.tsx — createOrder()
paymentMethod: orderData.paymentMethod,
paymentStatus: "paid",
transactionId: `TXN_${Date.now()}`,

// checkout/page.tsx — handlePlaceOrder()
setTimeout(() => { ... createOrder({...}); router.push(...) }, 1200);

// checkout/page.tsx — the card form
<input placeholder="4532 •••• •••• 8912"
       defaultValue="4532 8910 2341 8912" />
<input type="password" defaultValue="842" maxLength={4} />
```

The card fields are never read. The UPI step renders a static QR icon, not a real payment intent. There is no gateway, no charge, no webhook and no ledger anywhere in the codebase.

> **FIX**
> Rebuild the flow server-first: server computes the total, drafts a pending order, creates a gateway order, the customer pays on the gateway's hosted UI, and a signature-verified webhook is the only thing allowed to mark the order paid. Delete the card form entirely.

C-4 The JWT signing secret is published in the source tree

`apps/web/src/lib/auth/jwt.ts:4` · `apps/web/.env.example` · `.env.example`

```
const JWT_SECRET = process.env.JWT_SECRET ||
  "criation_jwt_master_secret_key_2026_artisan_dropship_platform_v2";
```

The fallback is a real, working secret, and the identical value sits in `apps/web/.env.example` — which is currently untracked but *not* git-ignored, so the next `git add .` commits it. Anyone with the repository can forge a valid admin token against any deployment that forgot to set the variable.

> **FIX**
> Throw at boot when `JWT_SECRET` is missing — `requireEnv()` already exists in `@criation/config`. Rotate the secret, replace both example files with a placeholder, and add secret scanning to pre-commit and CI.

C-5 Admin, seller and supplier consoles have no server-side authorization

`apps/web/src/app/admin/page.tsx:80` · `StoreContext.tsx:1002` · `no middleware.ts in the repo`

There is no `middleware.ts` anywhere, and no route handler ever checks a role. `/admin`, `/seller`, `/supplier` and `/dropship` are statically prerendered pages gated only by React state. The master key check accepts five hardcoded strings:

```
const cleanKey = pinOrKey.trim().toLowerCase();
if (cleanKey === "admin123" || cleanKey === "superadmin" ||
    cleanKey === "criation_admin" || cleanKey === "demo" ||
    cleanKey === "admin") { ... }
// and the failure toast reads: "Invalid superadmin master key. Try 'admin123'."
```

Editing `criation_user_v1` in devtools reaches the same place without any key at all.

> **FIX**
> Add `middleware.ts` verifying the JWT and role for every privileged page and API path, plus a per-handler `requireRole()` check. Delete the master-key mechanism.

C-6 Write endpoints are unauthenticated and mass-assign the request body

`apps/web/src/app/api/products/route.ts:57` · `apps/web/src/app/api/orders/route.ts:38`

```
// products/route.ts — no auth check anywhere in the file
const product = await Product.create({ ...body, slug: body.slug || slug });

// orders/route.ts — no auth check anywhere in the file
const order = await Order.create({ ...body, date: new Date() });
```

Unvalidated client JSON is spread straight into Mongoose. A stranger can insert catalog entries, or insert orders with `total: 1` and `paymentStatus: "paid"`. Note that `@criation/validation` already contains Zod schemas for exactly these shapes — the web app simply does not use them.

> **FIX**
> Require an authenticated session and the right role, parse the body with the existing Zod schemas, and allowlist fields explicitly. Money fields must be computed server-side and never accepted from a client.

C-7 Anonymous file upload with no type, size or quota limit

`apps/web/src/app/api/storage/upload/route.ts`

Anyone can POST any file of any size; it is written into `public/uploads` and served from your origin. There is no session check, no MIME allowlist, no byte cap and no rate limit. An `.html` or `.svg` upload becomes stored XSS on your own domain; a loop fills the disk. Because it is a plain multipart POST it is also CSRF-able despite `sameSite: lax`.

> **FIX**
> Require auth, allowlist `image/jpeg|png|webp|avif`, verify magic bytes rather than trusting `file.type`, cap at a few MB, re-encode through `sharp`, and store in object storage behind a CDN — not in `public/`.

C-8 A database outage becomes an authentication bypass

`apps/web/src/app/api/auth/login/route.ts:122 (catch block) · api/auth/me/route.ts:44`

When MongoDB is unreachable, login falls into a catch block that accepts any of five demo addresses — including `admin@criation.example` — on a hardcoded password, and issues a real signed JWT with whatever role the request body asked for:

```
} catch (dbError: any) {
  const DEMO_EMAILS = [..., "admin@criation.example"];
  if (password !== "password123" && password !== "admin123") { ...401... }
  const token = signToken({ ..., role: (role as Role) || "customer" });
  response.cookies.set(AUTH_COOKIE_NAME, token, COOKIE_OPTIONS);
```

`/api/auth/me` has the same shape: on a database error it returns a successful session built from the token alone. An attacker who can degrade the database — or who simply catches a real outage — gets admin.

> **FIX**
> Delete every fallback branch. A datastore failure must return `503`, never a credential. The same applies to the mock-catalog fallbacks in the product and order routes.

High

Not directly exploitable by an anonymous stranger, but each one either loses data, loses money, or removes a defence the platform needs before launch.

H-1 Identity is read from localStorage and never verified server-side

`apps/web/src/context/StoreContext.tsx:224`

The auth cookie is set correctly, but nothing validates it after login — `/api/auth/me` is never fetched anywhere in the codebase. On every page load the whole user object, including `role`, `isAdminVerified` and `walletBalance`, is rehydrated from `criation_user_v1`, which the user controls.

> **FIX**
> Restore the session server-side on request, and treat localStorage as a cache of non-authoritative UI state only.

H-2 Every first-time visitor arrives already signed in as a named user

`apps/web/src/lib/data/mockCatalog.ts:763`

```
export const initialUserProfile: UserProfile = {
  id: "usr_divyanshu",
  name: "Divyanshu Sharma",
  email: "mrdiv@criation.example",
  walletBalance: 750,  loyaltyPoints: 350,  tier: "Gold",
  isAuthenticated: true,
  ...
```

A brand new visitor sees someone else's name, address and wallet balance, and can check out without ever creating an account.

> **FIX**
> Default to a genuine guest: no name, no wallet, `isAuthenticated: false`. Keep the demo profile behind a seed script.

H-3 Order history exists only in the customer's browser

`apps/web/src/context/StoreContext.tsx:516, 271`

Orders are never sent to `/api/orders` — that endpoint exists but nothing calls it. Clearing site data, switching device or opening a private window destroys every order. You have no server record to fulfil, refund, dispute or account for, and no way to answer a customer asking where their parcel is.

> **FIX**
> Make order creation a server operation that returns the persisted order; the client renders only what the server confirmed.

H-4 The wallet is free money and coupons are validated client-side

`apps/web/src/context/StoreContext.tsx:474 (applyCoupon), 693 (cancelOrder), 820 (addWalletMoney)`

`addWalletMoney(amount)` credits the balance with no payment step at all. Cancelling a "paid" order refunds the full total into the wallet. Coupon codes are matched against a client-side array, so discount rules, minimum-order values and expiry are all editable by the customer. Subtotal, GST, shipping and total are likewise all computed in the browser.

> **FIX**
> Move pricing, coupon validation and every wallet mutation server-side into a ledger with immutable entries. The client sends item IDs and quantities; the server returns the price.

H-5 No security response headers at all

`apps/web/next.config.ts`

There is no `headers()` block: no Content-Security-Policy, HSTS, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy` or `Permissions-Policy`. The site can be framed for clickjacking, and there is no CSP backstop against injected script.

> **FIX**
> Add a `headers()` block with a nonce-based CSP, `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload`, `frame-ancestors 'none'`, `nosniff` and `Referrer-Policy: strict-origin-when-cross-origin`.

H-6 The image optimizer is an open proxy

`apps/web/next.config.ts:12`

```
images: {
  remotePatterns: [
    { protocol: "https", hostname: "images.unsplash.com" },
    { protocol: "https", hostname: "**" },
  ],
}
```

The wildcard lets anyone pass any URL through `/_next/image`, turning your server into a bandwidth relay and an SSRF-adjacent fetcher, billed to you.

> **FIX**
> Replace the wildcard with the explicit hosts you serve from, and set `images.dangerouslyAllowSVG: false`.

H-7 No rate limiting, lockout or bot protection anywhere

`all route handlers`

Login, register, upload and seed can each be called without limit. Credential stuffing, mass account creation and disk-filling uploads are unimpeded, and there is no CAPTCHA or device check on any sensitive action.

> **FIX**
> Per-IP and per-account rate limits, exponential lockout after failed logins, and a bot check on registration and password reset.

H-8 No password policy, no verification, no password reset

`apps/web/src/app/api/auth/register/route.ts:11`

Registration accepts a one-character password — the only check is that the field is present. Email and phone are never verified, so accounts can be created against addresses the user does not own. There is no forgot-password flow anywhere in the app: a customer who forgets their password is permanently locked out, and support has no recovery path.

> **FIX**
> Minimum 8–12 characters with a breach check, email verification before first order, and a token-based reset flow with short expiry and single use.

H-9 Uploads to the local filesystem will not work in production

`apps/web/src/app/api/storage/upload/route.ts:28`

The route writes to `process.cwd()/public/uploads`. On Vercel, Netlify or any container that redeploys, that directory is read-only or ephemeral — uploads either throw or vanish on the next deploy, and never appear on other instances.

> **FIX**
> Upload to object storage using presigned URLs, so the file never transits your server.

H-10 16 dependency advisories, 5 rated high

`npm audit at repository root`

1 low, 10 moderate, 5 high. The named advisories are listed in Appendix B. Build-time only today, but they gate any clean CI pipeline.

> **FIX**
> Run `npm audit fix`, bump Expo to a patched SDK, and add `npm audit` to CI as a blocking step.

Medium

Quality, correctness and compliance problems that will hurt once the platform is real, but do not create an open door today.

M-1 234 lint errors block any quality gate

`apps/web — 41 files · turbo run lint exits 1`

Mostly unused imports and `any`, but 11 of them are genuine correctness errors flagged by the React Compiler rules that are active in this project. Full breakdown in Appendix A. Until this is clean you cannot make lint a merge requirement, which is how the rest of this list stays fixed.

> **FIX**
> Run `npm run lint:fix`, then hand-fix the `react-hooks` errors and type the remaining `catch (e: any)` blocks as `unknown`.

M-2 Zero tests and no CI pipeline

`repository root — no test files, no .github/workflows`

Nothing prevents a regression in pricing, auth or checkout from reaching production.

> **FIX**
> Vitest for pricing, coupon and tax logic and every auth handler; Playwright for the browse → cart → checkout → order journey; a GitHub Actions workflow running lint, typecheck, test, build and audit on every PR.

M-3 Two backends and two type systems that disagree on price units

`backend/api/** · packages/types/src/product.ts:31 · apps/web/src/types/store.ts:47`

The Express service is unreachable dead weight — nothing calls it, its auth only accepts `demo@criation.example`, and its catalog is an in-memory array. Meanwhile the shared packages are used by admin and mobile but not by the app that matters. The two worlds also disagree on money:

```
// packages/types/src/product.ts
/** Price in the smallest currency unit, e.g. 129900 for ₹1,299.00 */
price: number;

// apps/web/src/types/store.ts
price: number; // In Rupees e.g. 399
```

Any future integration between them silently multiplies or divides every price by 100.

> **FIX**
> Pick one home for business logic and delete or fully adopt the other. Standardise on integer minor units everywhere and format at the edge.

M-4 No inventory control, and order numbers collide

`apps/web/src/context/StoreContext.tsx:526`

```
const randomSuffix = Math.floor(1000 + Math.random() * 9000);
const orderNumber = `CR-${randomSuffix}`;
```

Roughly a 1-in-9,000 collision per order, generated on the client where two customers can produce the same one — and the Mongoose schema marks `orderNumber` `unique`, so the second one fails to save. Stock is never decremented either, so the same item can be sold indefinitely.

> **FIX**
> Server-generated order numbers from a sequence or ULID, and an atomic stock decrement in the same transaction as order creation.

M-5 GST is a flat 5% and the invoice is not a compliant tax invoice

`apps/web/src/context/StoreContext.tsx:466` · `apps/web/src/lib/invoice/generateInvoice.ts`

```
const cartTax = useMemo(() => {
  const taxable = Math.max(0, cartSubtotal - cartDiscount);
  return Math.round(taxable * 0.05); // 5% GST — applied to every product
}, [cartSubtotal, cartDiscount]);
```

The generated document calls itself a Tax Invoice while carrying no GSTIN, no HSN/SAC codes, no place-of-supply and no IGST-versus-CGST/SGST split. For an Indian marketplace selling across states that is a compliance problem, not a formatting one.

> **FIX**
> Per-product HSN and GST rate on the product model, place-of-supply logic driving CGST+SGST versus IGST, seller GSTIN on the invoice, and sequential invoice numbering per financial year.

M-6 The AI concierge and AI tools are keyword matching

`apps/web/src/components/ai/AIChatWidget.tsx:73` · `apps/web/src/app/ai-tools/page.tsx`

The widget is a chain of `if (q.includes("gift"))` branches behind a 700 ms fake typing delay. Fine as a demo, but it is presented to users as an AI assistant and will fail on the first real question.

> **FIX**
> Either wire it to a real model through a server route — never exposing an API key to the browser — with a strict system prompt, rate limiting and a cost cap, or relabel it as a guided help menu.

M-7 Legal pages claim compliance the platform does not have

`apps/web/src/app/legal/[slug]/page.tsx:19 · app/checkout/page.tsx:140`

The privacy policy states that transactions run through "256-bit SSL encrypted, PCI-DSS Level 1 compliant payment gateways". There is no gateway. Checkout repeats the claim under a padlock icon. Publishing that to real customers is a consumer-protection exposure independent of the technical one.

> **FIX**
> Rewrite the legal pages against what the platform actually does once payments are live, and have them reviewed. Add the disclosures Indian e-commerce rules require: seller identity, grievance officer, return window, shipping timelines.

M-8 No transactional email or SMS

`whole app — no mail or SMS dependency in any package.json`

Nothing is ever sent to a customer: no order confirmation, no shipping notification, no OTP, no password reset, no invoice delivery. Notifications exist only as in-app toasts stored in the browser.

> **FIX**
> Add a transactional provider driven by server-side order events, with templates and delivery logging.

M-9 Product and order pages are client-only — no SSR, no SEO surface

`app/products/[id]/page.tsx:1 · app/orders/[id]/page.tsx:1 · no sitemap.ts or robots.ts`

Every product page is `"use client"` reading from context, so there is no per-product title, description, canonical URL, Open Graph image or Product structured data — and no sitemap or robots file. For a storefront that lives on search traffic, this is the difference between existing and not existing on Google.

> **FIX**
> Convert product and category pages to server components with `generateMetadata()`, add JSON-LD Product/Offer/BreadcrumbList, and add `sitemap.ts`, `robots.ts` and a web manifest.

M-10 Currency conversion uses hardcoded rates

`apps/web/src/context/StoreContext.tsx:38`

```
const CURRENCY_RATES = {
  INR: { symbol: "₹", rate: 1 },
  USD: { symbol: "$", rate: 0.012 },
  EUR: { symbol: "€", rate: 0.011 },
  GBP: { symbol: "£", rate: 0.0095 },
};
```

These constants were wrong the day after they were written, and a customer can be charged a different amount than they were shown.

> **FIX**
> Either sell in INR only with a clearly-labelled indicative conversion, or fetch daily rates server-side and freeze the rate onto the order at capture time.

M-11 Silent database fallbacks hide outages

apps/web/src/lib/db/mongodb.ts:39 · every route catch block

```
console.log(`[MongoDB] Connected successfully to: ${MONGODB_URI.split("@").pop()}`);
...
} catch (dbErr: any) {
  return NextResponse.json({ success: true, products: initialProducts, fallback: true });
}
```

A failed connection logs a warning and the route returns mock catalog data with `success: true`. Monitoring sees healthy 200s while the database is down, and customers browse a fictional catalog. The connect handler also logs part of the connection string.

> **FIX**
> Fail loudly — return 503, emit a structured error to an error reporter, and add a real health check. Remove the URI from the log line.

Low & housekeeping

L-1 The entire storefront is uncommitted

`git status on branch CT04`

Thirty-plus untracked directories including `src/app/api`, `src/components`, `src/context` and `src/lib`. All of this work exists on one machine only.

> **FIX**
> Commit in reviewable slices — but scrub `apps/web/.env.example` of the real secret first (see C-4), since it is not covered by the ignore rules.

L-2 A folder of personal WhatsApp photos sits at the repository root

`pp/ — 20+ .jpeg files, untracked and unreferenced`

> **FIX**
> If these are product photography, move them into `public/products` with sensible names. Otherwise delete the folder and add `pp/` to `.gitignore`.

L-3 The .gitignore is duplicated line-for-line

`.gitignore — two full copies of the same rule set concatenated`

Harmless today, but it makes the file hard to reason about and invites a rule being fixed in one half only. This is how `apps/web/.env.example` ended up un-ignored.

> **FIX**
> Deduplicate into one section-per-concern file.

L-4 No error boundaries, and one loading state for the whole app

`apps/web/src/app — only loading.tsx at root, no error.tsx or not-found.tsx`

A thrown error in any route renders the default Next.js error screen. Route-level loading states are missing, so navigation between heavy pages shows nothing.

> **FIX**
> Add `error.tsx` and `not-found.tsx` at the root and per segment, wired to your error reporter.

L-5 Accessibility has not been reviewed

`apps/web/src/app/checkout/page.tsx:440 and other interactive components`

Payment method tiles and several other controls are `<div onClick>` rather than buttons or radios, so they are unreachable by keyboard and unannounced to screen readers. There is no skip link, and focus order through the checkout steps has not been checked.

> **FIX**
> Semantic elements for anything interactive, a keyboard pass over the checkout, and axe in CI.

Appendix A — Lint errors in full

234 errors and 2 warnings across 41 files in `apps/web`. `turbo run lint` exits 1. All other nine workspaces are clean.

BY RULE

COUNT	RULE	CLASS
165	`@typescript-eslint/no-unused-vars`	Dead code
33	`@typescript-eslint/no-explicit-any`	Type safety
21	`react/no-unescaped-entities`	Markup
9	`react-hooks/purity`	**Correctness**
2	`react-hooks/set-state-in-effect`	**Correctness**
2	`@next/next/no-html-link-for-pages`	Routing
2	`prefer-const`	Style
1	`@next/next/no-location-assign-relative-destination`	Routing

THE 14 CORRECTNESS AND ROUTING ERRORS, INDIVIDUALLY

These are the ones that will not be fixed by `--fix` and represent real bugs under the React Compiler, which is enabled in `next.config.ts`.

FILE AND LINE	PROBLEM
`context/StoreContext.tsx:200`	`setThemeState()` called synchronously inside an effect — cascading renders
`context/StoreContext.tsx:284`	`Date.now()` and `Math.random()` called during render (2 errors)
`context/StoreContext.tsx:313`	`Date.now()` called during render — product ID generation
`context/StoreContext.tsx:351`	`Date.now()` called during render — sourcing request ID
`context/StoreContext.tsx:526`	`Math.random()` called during render — **order number generation**
`context/StoreContext.tsx:545`	`Date.now()` called during render — **transaction ID generation**
`context/StoreContext.tsx:548`	`Math.random()` called during render — courier tracking number
`components/ai/AIChatWidget.tsx:59`	`Date.now()` called during render — message ID
`components/ai/AIChatWidget.tsx:109`	`Date.now()` called during render — bot message ID
`components/layout/AppSidebar.tsx:75`	`setIsMobileOpen()` called synchronously inside an effect
`components/layout/AppSidebar.tsx:116`	`window.location.href` used for internal navigation instead of the router

FILE AND LINE	PROBLEM
components/layout/AppSidebar.tsx:172, 596	`<a href="/">` instead of `<Link />` — full page reload (2 errors)
api/products/route.ts:29	`products` declared `let`, never reassigned
lib/db/mongodb.ts:18	`cached` declared `let`, never reassigned

BY FILE — THE 20 WORST

ERRORS	FILE
22	src/components/layout/AppSidebar.tsx
13	src/app/seller/page.tsx
13	src/app/supplier/page.tsx
13	src/components/layout/Navbar.tsx
12	src/app/search/page.tsx
12	src/app/subscription/page.tsx
12	src/context/StoreContext.tsx
11	src/app/admin/page.tsx
11	src/app/page.tsx
9	src/app/account/page.tsx
9	src/app/dropship/page.tsx
9	src/app/orders/[id]/page.tsx
8	src/app/support/page.tsx
7	src/app/api/orders/route.ts
7	src/app/products/page.tsx
6	src/app/api/products/route.ts
6	src/components/ai/AIChatWidget.tsx
5	src/app/checkout/page.tsx
5	src/app/deals/page.tsx
5	src/app/orders/page.tsx

The remaining 21 files carry 1–4 errors each.

Appendix B — Dependency advisories

`npm audit` at the repository root: 16 total — 5 high, 10 moderate, 1 low. Named advisories below; the remainder are transitive dependents of these packages, almost all through the Expo toolchain in `apps/mobile`.

SEVERITY	PACKAGE	ADVISORY
HIGH	image-size	ICNS parser allows denial of service through an infinite loop — GHSA-w3rx-r6r6-pgpr
HIGH	image-size	JXL and HEIF parsers allow denial of service through infinite loops — GHSA-5p2g-fcmc-qvqq
MODERATE	uuid	Missing buffer bounds check in v3/v5/v6 when `buf` is provided — GHSA-w5hq-g745-h8pq
LOW	esbuild	Arbitrary file read when running the development server on Windows — GHSA-g7r4-m6w7-qqqr

AFFECTED DEPENDENCY CHAINS

`expo` → `@expo/cli`, `@expo/config`, `@expo/config-plugins`, `@expo/metro-config`, `@expo/prebuild-config`, `@expo/local-build-cache-provider`, `@expo/inline-modules`.

The `esbuild` advisory affects the local development server only. The `image-size` and `uuid` issues are reachable at build time. None of the four is currently exposed to end users, but all four block a CI gate that runs `npm audit`.